گزارش کار تمرین کامپیوتری سوم

سجاد تقی زاده

810102425

بخش اول

قایم کردن پیام توی عکس

1: مقدمه

پنهان‌نگاری یا همون Steganography یه جور هنره. علم مخفی کردن اطلاعات توی یه چیز دیگه است. مثلا قایم کردن یه پیام توی عکس. جوری که اصلا کسی نفهمه پیامی اون تو هست.

این با رمزنگاری فرق داره. توی رمزنگاری پیام معلومه ولی قفل شده. اینجا خود پیام رو قایم می‌کنیم.

توی این پروژه ما از عکس‌های سیاه سفید استفاده کردیم. روش کارمون هم دستکاری **بیت آخر** یا همون LSB بود. هر پیکسل یه عدده بین صفر تا ۲۵۵. بیت آخرش کمترین تاثیر رو توی روشنایی داره. مثلا اگه ۱۵۰ بشه ۱۵۱ چشم ما اصلا نمیفهمه.

هدف ما هم پیاده‌سازی همین کار بود. یعنی یه پیام متنی رو توی عکس قایم کنیم و بعد دوباره برش گردونیم. اونم با یه روش هوشمند برای انتخاب پیکسلا.

2: کد های مهم و توضیحات انها:

فایل createMapset.m

این فایل همون جدول یا دیکشنری رو میسازه که میگه هر حرفی چه کدی داره.

```
chars = ['a':'z', ' ', '.', ',', '!', '"', ';'];
```

- **توضیح :**اینجا ما ۳۲ تا کاراکتر مجازی که توی صورت سوال گفته شده بود رو تعریف می‌کنیم. اینا شامل حروف کوچیک انگلیسی و چندتا علامت خاص هستن.

```
Mapset = cell(2, 32);
```

- **توضیح :**یه آرایه سلولی خالی با ۲ ردیف و ۳۲ ستون می‌سازیم. ردیف اول برای خود کاراکترها مثل 'a' و ردیف دوم برای کدهای باینری‌شون (مثل '00000').

```
Mapset{2, i} = dec2bin(i-1, 5);
```

- **توضیح :**این خط توی حلقه forقرار داره. تابع dec2binمیاد عدد i-1رو (که از ۰ تا ۳۱ میره) به باینری تبدیل می‌کنه. پارامتر 5هم مجبورش می‌کنه که حتما ۵ رقمی باشه (مثلا به جای '1' بده '00001'). این کد باینری رو توی ردیف دوم Mapsetذخیره می‌کنه.

فایل coding.m

این پیامی که بهش میدیم رو توی عکس جاسازی می‌کنه.

```
message_with_terminator = [message, ';'];
```

- **توضیح :**قبل از هر کاری میایم کاراکتر (;) رو به آخر پیامی که گرفتیم می‌چسبونیم. این یه نشونه است که به تابع (decoding)میگه پیام اینجا تموم شده.

```
binary_string = [binary_string, Mapset{2, idx}];
```

- **توضیح :**این خط توی یه حلقه forهست. دونه دونه حروف پیام رو (شامل اون سمی‌کالن آخری) برمیداره، کد ۵ بیتی‌ش رو از Mapsetپیدا می‌کنه و همه رو به هم می‌چسبونه تا یه رشته باینری خیلی طولانی درست بشه.

```
if var(double(block(:))) > threshold
```

- **توضیح :**این مهم‌ترین بخش کار ماست. عکس رو به بلوک‌های کوچیک (مثلا ۵ در ۵) تقسیم می‌کنه و واریانس هر بلوک رو حساب می‌کنه. اگه واریانس از یه حدی بیشتر بود، یعنی اون بلوک شلوغه و برای قایم کردن داده مناسبه.

```
suitable_pixel_indices = [suitable_pixel_indices; linear_indices];
```

- **توضیح :**اگه بلوک مناسب بود، آدرس تمام پیکسل‌های توی اون بلوک رو توی این متغیر ذخیره می‌کنه. تهش ما یه لیست بلند از آدرس تمام پیکسل‌های مناسب داریم.

```
if num_bits_to_hide > length(suitable_pixel_indices)
```

- **توضیح :**یه چک ساده ولی حیاتی. میبینه آیا تعداد بیت‌های پیام ما از تعداد پیکسل‌های مناسبی که پیدا کردیم بیشتره یا نه. اگه بیشتر بود، یعنی جا کم داریم و ارور میده.

```
encoded_image(pixel_idx) = bitset(original_pixel_value, 1, bit_to_hide);
```

- **توضیح :**این خط خود عمل جاسازی رو انجام میده. تابع bitsetمیره سراغ پیکسلی که آدرسش رو داریم(pixel_idx) ، بیت شماره 1اون یعنی همون LSB یا کم‌اهمیت‌ترین بیت رو برمیداره و مقدارش رو برابر با بیت پیام ما (bit_to_hide) که ۰ یا ۱ هست، قرار میده.

فایل decoding.m

این تابع عکس رو میگیره و پیام رو میکشه بیرون.

```
if var(double(block(:))) > threshold
```

- **توضیح :**این خط *دقیقا* کپی همون خط توی coding.mهست. این خیلی مهمه! کدگشا باید *دقیقا* از همون استراتژی (همون اندازه بلوک و همون thresholdاستفاده کنه تا بتونه *دقیقا* همون پیکسل‌هایی که داده توشون قایم شده رو به همون ترتیب پیدا کنه.

```
suitable_pixel_indices = [suitable_pixel_indices; linear_indices];
```

- **توضیح :**باز هم مثل coding.m، لیست آدرس پیکسل‌های مناسب رو بازسازی می‌کنه.

```
lsb = bitget(pixel_value, 1);
```

- **توضیح :**این برعکس bitsetهست. تابع bitgetمیره سراغ پیکسل مورد نظر و بیت شماره 1 اون همون LSB رو میخونه و مقدارش رو برمی‌گردونه.

```
binary_string = [binary_string, num2str(lsb)];
```

- **توضیح :**بیت‌هایی که ازLSB ها خونده رو یکی یکی به هم می‌چسبونه تا اون رشته باینری طولانی دوباره ساخته بشه.

```
if strcmp(five_bits, '11111')
```

- **توضیح :**بعد از ساختن رشته باینری، میاد ۵ تا ۵ تا جدا می‌کنه. اینجا چک می‌کنه که آیا این ۵ بیت برابر با '11111'هست یا نه. این کد همون (;)یا نشانگر پایان پیامه.

```
break;
```

- **توضیح :**به محض اینکه به کد '11111'برسه، میفهمه پیام تموم شده و از حلقه forمیپره بیرون. اینجوری داده‌های اضافی یا آشغال رو به عنوان پیام نمیخونه.

```
decoded_message = [decoded_message, Mapset{1, idx}];
```

- **توضیح :**اگه کد '11111'نبود، میره توی Mapset(این بار توی ردیف دوم) دنبال اون ۵ بیت میگرده و کاراکتر معادلش رو (از ردیف اول) پیدا می‌کنه و به پیام نهایی اضافه می‌کنه.

جواب تمرین ۳

- **آیا تغییرات مشخص است؟** خیر تغییرات در تصویر اصلاً با چشم غیر مسلح مشخص نیستند.
- **چرا؟** چون ما فقط **بیت (LSB)** هر پیکسل را دستکاری کردیم . این تغییر، مقدار روشنایی پیکسل را حداکثر ۱ واحد از ۲۵۶ سطح روشنایی ممکن تغییر می‌دهد که این تفاوت برای چشم انسان قابل تشخیص نیست. به علاوه ما این تغییرات را فقط در بلوک‌های شلوغ و پر جزئیات تصویر اعمال کردیم که باعث می‌شود این تغییرات جزئی کاملا در میان بافت خود تصویر پنهان شوند.

جواب تمرین ۵

- **آیا باز هم قادر به رمزگشایی خواهیم بود؟** خیر به احتمال بسیار زیاد پیام به درستی رمزگشایی **نمی‌شود** و خراب یا ناقص خواهد بود.
-
- **توضیح:**
  1. روش ما به این وابسته است که LSB پیکسل‌های انتخاب‌شده *دقیقاً* همان چیزی باشند که ما قرار داده‌ایم.
  2. نویز به صورت ناخواسته و تصادفی مقادیر پیکسل‌ها را تغییر می‌دهد.

3. این تغییرات تصادفی به راحتیLSB ها را (از ۰ به ۱ یا برعکس) عوض می‌کنند.
4. خراب شدن حتی یک بیت در یک گروه ۵ بیتی، باعث خوانده شدن یک کاراکتر اشتباه می‌شود. بدتر از آن، اگر نویز کد '11111' (مربوط به کاراکتر پایان ';') را خراب کند، کدگشا نمی‌فهمد پیام کجا تمام شده و اطلاعات بی‌معنی را ادامه می‌دهد.

خروجی ها:

```
Command Window
Mapset created 3 entries:
    {'a'    }    {'b'    }    {'c'    }
    {'00000'}    {'00001'}    {'00010'}

Encoding message "signal"
Plotted original vs encoded image.
Decoding image...
Decoded message: signal
Success
>>
```

بخش دوم

ردیابی شیء متحرک در ویدیو

۱ .مقدمه

هدف بخش دوم پروژه نوشتن یه اسکریپت برای ردیابی هواپیما توی ویدیوهای IR ردیابی یعنی یه باکس دور هواپیما بکشیم و این باکس با حرکت هواپیما حرکت کنه. یه چالش مهم هم این بود که اگه هواپیما نزدیک شد و بزرگ شد باکس ما هم باید هوشمند باشه و بزرگ بشه.

توی این گزارش استراتژی مبتنی بر **حرکت** رو توضیح میدیم. توی این روش ما فرض می‌کنیم دوربین ثابته و هواپیما "متحرک‌ترین" چیز توی تصویره.

۲ .توضیح کدهای مهم

```
MOTION_THRESHOLD ; MIN_MOTION_AREA ;
```

- **توضیح** :این دو تا پارامترهای اصلی ما برای تنظیم ردیاب هستن. اولی میگه چقدر تغییرات بین دو فریم رو حرکت حساب کنیم. دومی میگه اون لکه متحرک حداقل چقدر باید بزرگ باشه تا به حساب بیاد و نویز حساب نشه.

```
previousFrameGray = rgb2gray(previousFrame);
```

- **توضیح** :برای شروع کار ما فریم اول رو می‌خونیم و به عکس سیاه سفید تبدیلش می‌کنیم. این میشه مبنای ما برای مقایسه با فریم بعدی.

```
diffImage = abs(currentFrameGray - previousFrameGray);
```

- **توضیح** :این خط اصل کار ماست. میاد فریم فعلی رو از فریم قبلی کم می‌کنه. هرجایی که چیزی حرکت کرده باشه نتیجه این تفریق یه عدد غیر صفره.

```
motionMask = diffImage > MOTION_THRESHOLD;
```

- **توضیح** :اینجا میایم روی اون عکسِ تفریق شده یه فیلتر میذاریم. هر پیکسلی که مقدارش از آستانه حرکت (مثلا ۲۵) بیشتر بوده باشه رو ۱ (سفید) می‌کنه و بقیه رو ۰ (سیاه). نتیجه یه عکس با لکه‌های سفیده که جاهای متحرک رو نشون میده.

```
cleanedMask = imopen(motionMask, se_noise); cleanedMask = imclose(cleanedMask, se_group);
```

- **توضیح :**ماسک حرکتی که به دست آوردیم معمولا پر از نویز و نقطه‌های ریزه. با این دو دستور میایم تمیزکاریش می‌کنیم imopen .نویزهای ریز رو حذف می‌کنه و imcloseلکه‌های نزدیک به هم رو به هم می‌چسبونه تا هواپیما یه تیکه بشه.

```
cc = bwconncomp(cleanedMask); stats = regionprops(cc, 'Area', 'BoundingBox');
```

- **توضیح :**بعد از تمیزکاری ما با bwconncompتمام لکه‌های سفید (اشیای متحرک) رو پیدا می‌کنیم. بعد regionpropsمیاد مشخصات هر لکه مثل مساحت و مختصات باکس دورش رو برامون درمیاره.

```
[maxArea, maxIndex] = max([stats.Area]);
```

- **توضیح :**اینجا ما فرض کردیم هواپیما بزرگترین شیء متحرک توی تصویره. پس میایم مساحت همه لکه‌ها رو نگاه می‌کنیم و بزرگترینشون رو پیدا می‌کنیم.

```
if maxArea > MIN_MOTION_AREA
```

- **توضیح :**این یه فیلتر نهاییه. چک می‌کنه که آیا اون بزرگترین لکه‌ای که پیدا کردیم از حداقل مساحت بزرگتره. اگه آره پس حتما خودشه و نویز نیست.

```
outputFrame = insertShape(outputFrame, 'Rectangle', bbox, 'Color', 'green', ...);
```

- **توضیح :**این خط آخر کاره. میاد روی فریم اصلی یه مستطیل (Rectangle) با رنگ سبز(green) و ضخامت ۳ دور اون لکه بزرگه که پیدا کردیم می‌کشه. چون این باکس از regionpropsمیاد اندازه‌اش دقیقا متناسب با اندازه لکه حرکتیه پس مشکل تغییر سایز هم حل میشه.

```
previousFrameGray = currentFrameGray;
```

- **توضیح :**آخر حلقه whileما فریم فعلی رو توی متغیر فریم قبلی می‌ریزیم. اینجوری توی دور بعدی حلقه فریم جدید با فریم فعلی مقایسه میشه و این روند تا آخر ویدیو ادامه پیدا می‌کنه.

۳ .خلاصه

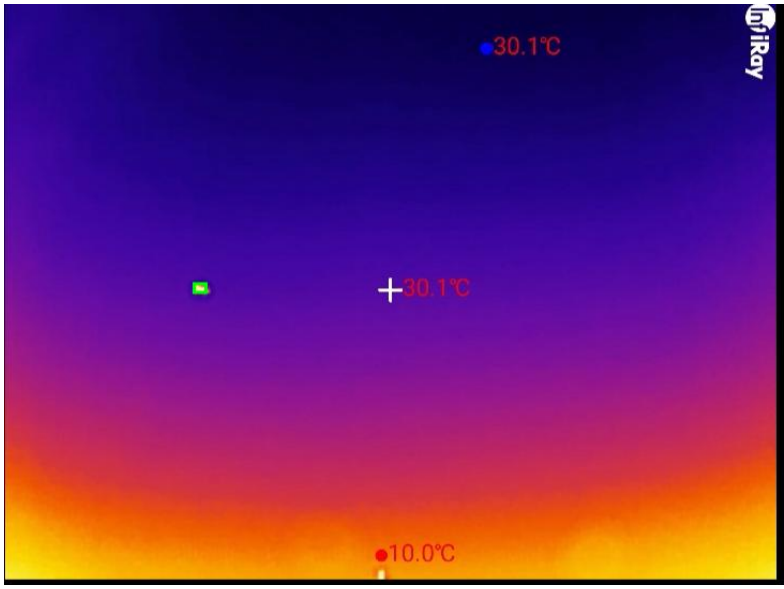
روند کار بخش دوم اینجوری بود:

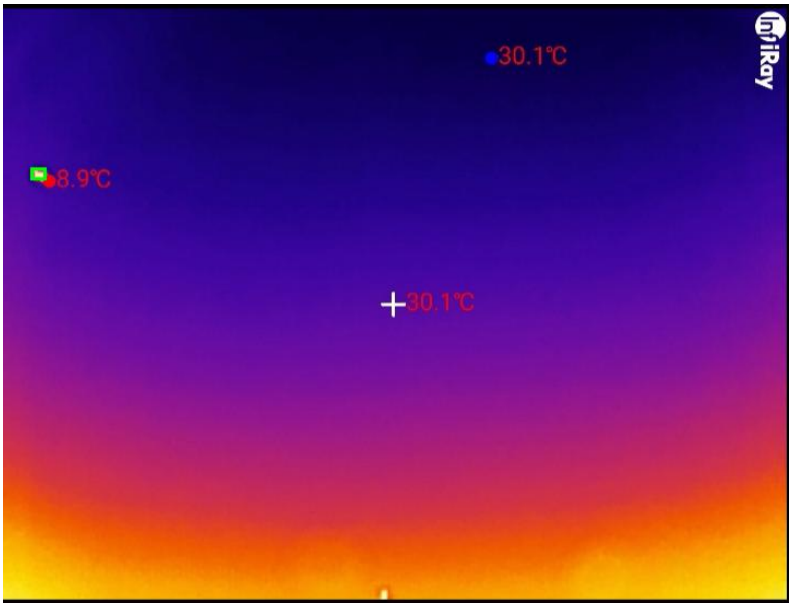
1. ویدیو رو از کاربر گرفتیم.
2. فریم اول رو به عنوان مبنا نگه داشتیم.
3. بعد توی یه حلقه تا آخر ویدیو فریم به فریم جلو رفتیم.
4. هر فریم جدید رو از فریم قبلیش کم کردیم تا حرکت‌ها مشخص بشن.
5. عکسی که از تفریق به دست اومد رو فیلتر و تمیزکاری کردیم تا نویزهاش بره.
6. بزرگترین لکه متحرک رو پیدا کردیم و فرض کردیم این همون هواپیماست.
7. اگه لکه به اندازه کافی بزرگ بود یه باکس سبز دورش کشیدیم.
8. فریم رو نشون دادیم و رفتیم سراغ فریم بعدی.

اینجوری یه ردیاب اتوماتیک ساختیم که خودش متحرک‌ترین چیز رو پیدا می‌کنه و دنبالش می‌کنه.

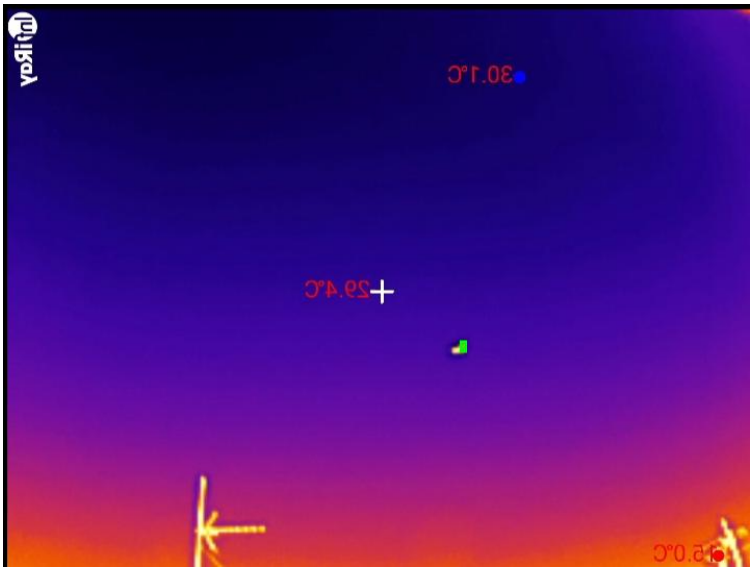
خروجی ها:

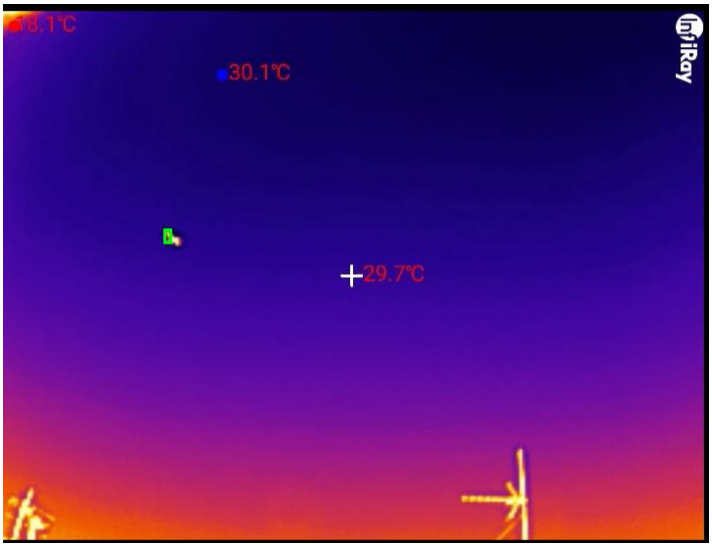
<u>نمونه ترک در ویدیو1:</u>





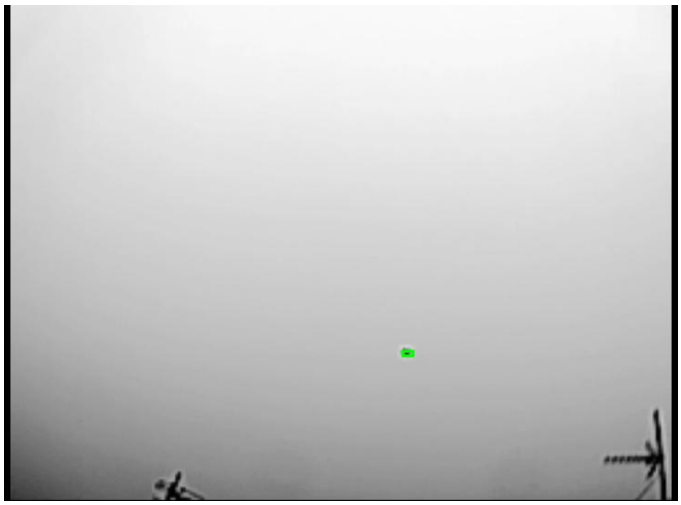
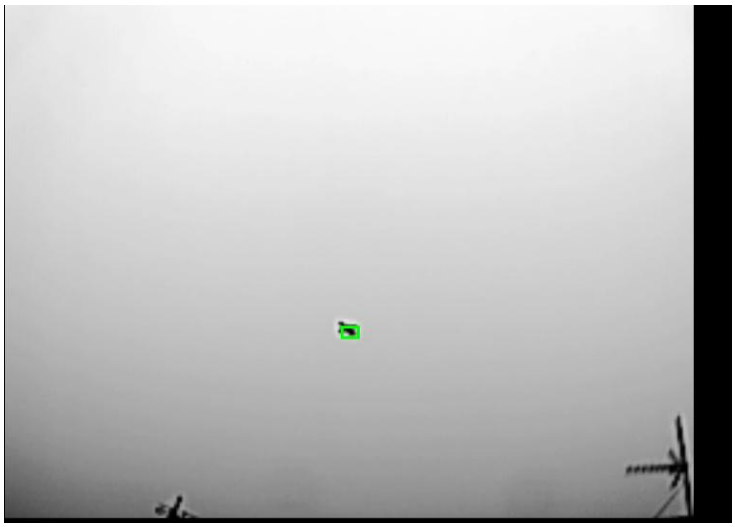
نمونه ترک در ویدیو 2:





نمونه ترک در ویدیو3:

نمونه ترک در ویدیو 4:



نمونه ترک در ویدیو 5: